

Building LLMs from Scratch

Generated by Bookify

Table of Contents

Introduction

Introduction & Fundamentals

- | Introduction to Large Language Models
- | The Significance of "Large" in LLMs
- | The Hierarchy of AI Terminologies
- | The Secret Sauce: The Transformer Architecture
- | The Mechanism of Self-Attention
- | Encoder vs. Decoder
- | Stages of Building an LLM
- | Stage 1: Pre-training (The Foundational Phase)
- | Stage 2: Fine-tuning (The Specialization Phase)
- | Learning Paradigms: Zero-Shot vs. Few-Shot
- | Emergent Behavior: The "Magic" of Scale
- | Technical Implementation Roadmap
- | Data Preparation and Sampling
- | Architecture Assembly
- | Training and Evaluation
- | Summary

Data Preprocessing & Tokenization

- | Context: The Foundation of Large Language Models
- | Step 1: Tokenization Mechanisms
- | Word-Based Tokenization
- | Character-Based Tokenization
- | Byte Pair Encoding (BPE)
- | Step 2: Vocabulary and Token IDs
- | Building the Vocabulary
- | Special Context Tokens
- | Step 3: Creating Input-Target Pairs
- | The Sliding Window Approach
- | DataLoaders and Batching

- | Step 4: Token Embeddings
- | Why Vectors?
- | The Embedding Layer as a Lookup Table
- | Step 5: Positional Embeddings
- | Absolute vs. Relative Positioning
- | Generating the Final Input
- | Summary of the Preprocessing Pipeline

The Attention Mechanism

- | Context and Motivation
- | Simplified Self-Attention: The Purest Form
- | From Embeddings to Context
- | The Role of the Dot Product
- | Implementation Step-by-Step
- | Self-Attention with Trainable Weights
- | The Query, Key, and Value Analogy
- | Scaled Dot-Product Attention
- | Causal Self-Attention
- | The Masking Mechanism
- | Dropout in Attention
- | Multi-Head Attention (MHA)
- | The Wrapper Approach vs. Weight Splitting
- | Mathematical Flow of Weight Splitting
- | Implementation Detail: The Multi-Head Class
- | Advanced Considerations
- | Registering Buffers
- | Numerical Stability
- | Variance and Scaling

LLM Architecture & Components

- | Context and Overview of the LLM Architecture
- | The Transformer Block: A Deep Dive
- | Structural Components
- | Layer Normalization (LayerNorm)

- | GELU Activation and the Feed-Forward Network
- | Shortcut Connections
- | Assembling the Full GPT Architecture
- | The Configuration (GPT-2 124M)
- | The Forward Pass Workflow
- | Parameter Count and Weight Tying
- | Implementation: The GPT Model Class
- | Generating Text: From Logits to Tokens
- | The Iterative Generation Loop
- | Greedy Search vs. Sampling
- | Worked Example: Next Token Prediction
- | Summary

Pre-training & Model Evaluation

- | Context and Overview
- | Measuring the LLM Loss Function
- | Inputs and Targets
- | From Logits to Probabilities
- | Cross-Entropy Loss and Negative Log-Likelihood
- | Evaluating Performance on Real Datasets
- | Data Splitting and Loading
- | The Evaluation Loop
- | The Pre-training Loop Implementation
- | Parameter Scale
- | The Training Step
- | Monitoring and Overfitting
- | Advanced Decoding Strategies
- | Temperature Scaling
- | Top-k Sampling
- | Loading Pre-trained Weights from OpenAI
- | Downloading and Extracting Parameters
- | Weight Mapping and Assignment
- | Saving and Loading Model States

| Saving the Model and Optimizer

| Loading the Checkpoint

| Summary

Classification Fine-tuning

| Introduction to Classification Fine-tuning

| Data Preparation and Pre-processing

| Dataset Analysis and Balancing

| Data Splitting

| Tokenization and Data Loading

| The Padding Challenge

| Implementing the PyTorch Dataset

| DataLoaders

| Architectural Modifications for Classification

| The Classification Head

| Freezing vs. Fine-tuning

| Extracting the Last Token

| The Training Loop and Loss Function

| Cross-Entropy Loss

| Optimization with AdamW

| Evaluation Metrics

| Model Inference and Deployment

| The Inference Pipeline

| Saving and Loading

| Summary

Instruction Fine-tuning & Conclusion

| Opening Context

| 1. The Necessity of Instruction Fine-tuning

| 1.1 Bridging the Gap to Personal Assistants

| 1.2 Supervised Instruction Fine-tuning (SFT)

| 2. Dataset Preparation and Prompt Formatting

| 2.1 The Alpaca Prompt Style

| 3. Advanced Data Batching Mechanics

3.1	The Five-Step Batching Workflow
3.2	The Logic of the "Ignore Index"
4.	Implementing DataLoaders
4.1	Custom Collate Function
5.	Loading Pre-trained Weights
5.1	Model Configuration
5.2	The "Knowledgeable State"
6.	The Fine-tuning Training Loop
6.1	Optimization Strategy
6.2	Loss Convergence
7.	Evaluating Generative Performance
7.1	Frameworks for Evaluation
8.	Conclusion and Summary of the LLM Build
8.1	The Journey Recap

Conclusion

Glossary

References & Resources

	Introduction & Fundamentals
	Data Preprocessing & Tokenization
	The Attention Mechanism
	LLM Architecture & Components
	Pre-training & Model Evaluation
	Classification Fine-tuning
	Instruction Fine-tuning & Conclusion

Introduction

The era of Large Language Models (LLMs) has transformed artificial intelligence from a niche academic pursuit into a cornerstone of modern technology, yet for many developers, these models remain "black boxes" hidden behind complex APIs. This book is designed to demystify that technology by guiding you through the process of building a fully functional LLM from the ground up. We begin with the mathematical fundamentals and the evolution of natural language processing before moving directly into the practical engineering required to prepare raw text for machine consumption. You will learn how to implement sophisticated data

preprocessing pipelines and build custom tokenizers, ensuring you understand exactly how human language is translated into the numerical representations that neural networks require.

At the heart of this journey is a deep dive into the Attention Mechanism, the breakthrough innovation that allows models to understand context and long-range dependencies in text. You will not just learn the theory of "Attention is All You Need," but you will code the architecture piece by piece, assembling the layers, normalization components, and feed-forward networks that constitute a modern transformer. Once the architecture is complete, the book transitions into the rigorous world of pre-training and model evaluation. Here, you will learn how to orchestrate the training process, manage computational resources, and use quantitative metrics to ensure your model is truly learning the underlying patterns of language.

This book is written for software engineers, data scientists, and AI enthusiasts who possess a foundational understanding of Python and basic machine learning concepts but want to move beyond simply using pre-built libraries. It is designed as a hands-on laboratory; the best way to use this text is to follow along with the code implementations, experimenting with hyperparameters and observing how architectural changes affect performance. By the final chapters, you will move from general pre-training to specialized applications, mastering the art of fine-tuning your model for specific tasks like sentiment classification and instruction following.

By the time you reach the conclusion, you will have transitioned from a consumer of AI to a creator. You will possess a comprehensive mental map of the entire LLM lifecycle, from a raw text file to a sophisticated, instruction-aligned assistant. More importantly, you will have developed the intuition necessary to debug, optimize, and innovate within the rapidly evolving landscape of generative AI. Whether you aim to build proprietary models for your organization or contribute to the open-source community, this book provides the technical blueprint and the practical confidence to build the future of intelligent software.

Introduction & Fundamentals

Introduction to Large Language Models

The landscape of Artificial Intelligence has undergone a seismic shift in recent years, moving from specialized, rule-based systems to generalized, generative architectures. At the heart of this transformation lies the Large Language Model

(LLM). To understand LLMs, one must view them through two lenses: their structural identity as deep neural networks and their functional identity as sophisticated text processors.

Technically, an LLM is a neural network designed to understand, generate, and respond to human-like text [1]. While the term "neural network" may sound abstract, it refers to a mathematical architecture inspired by the biological circuitry of the human brain. These networks consist of layers of "neurons"—computational units that process input data through weighted connections. When these layers are stacked together, the model gains the ability to identify complex patterns in data, ranging from image detection to text generation [2].

The evolution of this field is best illustrated by comparing early attempts at natural language processing (NLP) with modern systems. In the 1960s, the ELIZA chatbot functioned as a primitive therapist. However, ELIZA was not "intelligent" in the modern sense; it relied on simple pattern matching. If a user mentioned "large language models," ELIZA might simply repeat the phrase back to the user as a question, regardless of the context [3]. In contrast, modern LLMs like GPT-4 demonstrate a profound grasp of context, enabling them to plan relaxing days, solve complex problems, and maintain coherent multi-turn conversations [4].

The Significance of "Large" in LLMs

A common question in the field is why these models are specifically designated as "large" rather than simply "language models." The distinction lies in the scale of their parameters. Parameters are the internal variables that the model adjusts during its training process to learn from data.

Historically, AI models were relatively small. However, the advent of the LLM era saw an exponential explosion in scale. For example, GPT-3 operates with 175 billion parameters [5].

Table 1: *Evolution of GPT Parameter Counts*

Model Version	Parameter Count	Key Characteristic
GPT-1	117 Million	Initial generative pre-training
GPT-2	1.5 Billion	Zero-shot task transfer capabilities
GPT-3	175 Billion	Massive scale; few-shot learning
GPT-4	Trillions (Estimated)	Multimodal capabilities; high reasoning

This massive scale allows the model to capture a vast knowledge base from its training data. GPT-3's 175 billion parameters enable it to store information from a massive corpus of data, including hundreds of billions of words from the internet and books, and recall that information to generate contextually relevant text [\[6\]](#).

The Hierarchy of AI Terminologies

As the field grows, technical terminologies often become blurred. To understand LLMs, one must recognize the hierarchy of these concepts:

1. **Artificial Intelligence (AI):** The broadest umbrella, encompassing any machine or system that mimics human intelligence.
2. **Machine Learning (ML):** A subset of AI where machines learn and adapt based on data interactions.
3. **Deep Learning (DL):** A subset of ML that specifically utilizes deep neural networks. This technology powers image recognition and handwritten digit classification [\[7\]](#).
4. **Large Language Models (LLMs):** A specific application of deep learning focused on text. While DL can handle images and audio, LLMs are specialized for linguistic sequences [\[8\]](#).
5. **Generative AI (GenAI):** A broader category that overlaps with LLMs and DL. While LLMs generate text, GenAI includes models that generate images, video, and audio [\[9\]](#).

The Secret Sauce: The Transformer Architecture

The primary catalyst for the current LLM revolution was the introduction of the Transformer architecture in the 2017 paper "Attention Is All You Need" [\[10\]](#). Before Transformers, NLP relied on Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. Transformers represented a significant advancement because they utilize a self-attention mechanism to capture long-range dependencies in a sentence [\[11\]](#).

The Mechanism of Self-Attention

The "Secret Sauce" of the Transformer is the self-attention mechanism. This allows the model to weigh the importance of different words in a sentence relative to one another. Self-attention enables the model to look at the entire sentence and assign importance to relevant words, regardless of how far apart they are in the text [\[12\]](#).

Encoder vs. Decoder

The original Transformer was designed for tasks like machine translation and consisted of two main components: * **The Encoder:** Processes the input text [\[13\]](#). * **The Decoder:** Generates the output text [\[13\]](#).

Interestingly, the GPT (Generative Pre-trained Transformer) lineage diverged from this original design. GPT models are "decoder-only" architectures. They do not have a separate encoder; instead, they are optimized to predict the next word in a sequence [\[14\]](#).

Stages of Building an LLM

Building an LLM involves two primary stages: Pre-training and Fine-tuning [\[15\]](#).

Stage 1: Pre-training (The Foundational Phase)

Pre-training involves training a model on a massive, diverse dataset to teach it the fundamental structures of language. This stage produces what is known as a "Base Model" [\[16\]](#).

The data used for pre-training is typically "unlabeled text." This means the model uses a method of unsupervised learning where it learns to predict the next word in a sentence. Because the true answer is already in the text, the model can check its own work and adjust its internal parameters to minimize error [\[17\]](#).

The scale of pre-training is immense. GPT-3 was trained on a corpus including: * **Common Crawl (60%):** A massive repository of web page data [\[6\]](#). * **WebText2:** High-quality outbound links from Reddit [\[18\]](#). * **Books & Wikipedia:** Structured, high-quality literary and factual data [\[19\]](#).

The cost of this stage is a significant barrier. Pre-training GPT-3 required approximately \$4.6 million in computational expenses [\[20\]](#).

Stage 2: Fine-tuning (The Specialization Phase)

While a pre-trained model is broad, it often lacks specific behavior. Fine-tuning is the process of refinement by training on a narrower dataset specific to a particular task [\[21\]](#).

There are different types of fine-tuning: 1. **Instruction Fine-tuning:** Training the model to follow specific commands, transforming a base model into a helpful

assistant [\[22\]](#). 2. **Classification Fine-tuning:** Training the model for specific labels, such as "Spam" vs. "Not Spam" [\[23\]](#).

Learning Paradigms: Zero-Shot vs. Few-Shot

One of the most remarkable aspects of LLMs is their ability to perform tasks they weren't explicitly trained for:

- **Zero-Shot Learning:** The model is given a task description but no examples [\[24\]](#).
- **One-Shot Learning:** The model is given the task description and exactly one example [\[25\]](#).
- **Few-Shot Learning:** The model is given several examples to establish a pattern. This is the paradigm where GPT-3 excels, as highlighted in the paper "Language Models are Few-Shot Learners" [\[26\]](#).

Emergent Behavior: The "Magic" of Scale

As LLMs grew in size, researchers noticed a phenomenon called **Emergent Behavior**. This refers to the model's ability to perform complex tasks that it was never explicitly programmed to do [\[27\]](#).

This behavior is "emergent" because it only appears once the model reaches a certain threshold of scale. This suggests that at a certain scale, the model is developing a primitive form of reasoning [\[28\]](#).

Technical Implementation Roadmap

Building an LLM from scratch requires a systematic approach to data and architecture:

Data Preparation and Sampling

Before training, text must be processed. This involves: 1. **Tokenization:** Breaking sentences into smaller units (tokens) [\[29\]](#). 2. **Vector Embedding:** Mapping tokens into a mathematical space [\[30\]](#). 3. **Positional Encoding:** Adding a signal to each token to tell the model where that word sits in the sequence [\[31\]](#).

Architecture Assembly

The model is built by understanding the basic mechanism behind the large language model, including the attention mechanism and how to stack different layers [\[32\]](#).

Training and Evaluation

The final hurdle is the training loop, which involves feeding the model data and using backpropagation to adjust weights to reduce loss [\[33\]](#).

Summary

The journey of building a Large Language Model from scratch begins with a fundamental shift in perspective. We have moved from the rule-based chat systems of the 1960s to the massive, autoregressive Transformer architectures of today. Key takeaways include the critical role of the self-attention mechanism, the distinction between "decoder-only" GPT architectures and original Transformers, and the two-stage lifecycle of pre-training and fine-tuning.

Data Preprocessing & Tokenization

Context: The Foundation of Large Language Models

Building a Large Language Model (LLM) from scratch is a multi-stage engineering endeavor. The process typically proceeds through three distinct phases: understanding the basic mechanisms (data preparation, attention, and architecture), pre-training a foundational model, and fine-tuning for specific applications [\[34\]](#).

At the very beginning of this journey lies the data preprocessing pipeline. While much of the excitement in AI centers on model architectures and training loops, the performance of an LLM is fundamentally constrained by how raw text is transformed into a format a neural network can process. LLMs are, at their core, mathematical engines; they do not "read" text in the human sense. Instead, they

operate on numerical vectors. This chapter details the pipeline required to convert documents into the structured, numerical input-target pairs necessary for model training [\[35\]](#).

Step 1: Tokenization Mechanisms

Tokenization is the process of breaking down raw text into smaller units called tokens. These units can be words, subwords, or individual characters [\[36\]](#).

Word-Based Tokenization

In its simplest form, tokenization involves splitting text by whitespace and punctuation. Using Python's `re` (regular expression) module, we can define a splitting scheme that treats punctuation as separate tokens [\[37\]](#).

```
import re

text = "Hello, world. This is a test."
# Split by whitespace (\s) and punctuation characters
result = re.split(r'([, . : ; ? _ ! " ( ) \ ' ] | -- | \s)', text)
# Remove whitespace-only tokens
tokens = [item.strip() for item in result if item.strip()]
```

While intuitive, word-based tokenization suffers from significant drawbacks: 1. **Vocabulary Size:** A word-based vocabulary would require a massive embedding matrix to account for every unique word in a language [\[38\]](#). 2. **Out-of-Vocabulary (OOV) Words:** If the model encounters a word not seen during training, it fails to recognize it. To mitigate this, developers often add a special `<|unk|>` (unknown) token to the vocabulary [\[39\]](#). 3. **Morphological Blindness:** Words like "boy" and "boys" are treated as unrelated entities, losing the semantic connection of the root word [\[40\]](#).

Character-Based Tokenization

At the other extreme, we can treat every character as a token. This results in a very small vocabulary and eliminates the OOV problem [\[41\]](#). However, it forces the model to learn the meaning of words from scratch and significantly increases the sequence length the model must process [\[42\]](#).

Byte Pair Encoding (BPE)

Modern LLMs like GPT-2 and GPT-3 utilize a middle-ground approach called Byte Pair Encoding (BPE). BPE is a subword tokenization scheme that ensures frequently used words are represented as single tokens, while rare words are broken down into subword units [43].

The BPE Algorithm Process: 1. Initialize the vocabulary with individual characters. 2. Identify the most frequent pair of adjacent tokens in the training corpus. 3. Merge this pair into a new, single token and add it to the vocabulary. 4. Repeat the process for a fixed number of iterations [44].

For example, if the word "older" is rare but "old" and "er" are common, BPE will represent "older" as two tokens: `["old", "er"]`. This allows the model to generalize to new words by understanding their components [45].

Table 2: Comparison of Tokenization Strategies

Strategy	Vocabulary Size	OOV Handling	Sequence Length
Word-based	Large	Poor (needs <code>< unk ></code>)	Short
Character-based	Very Small	Excellent	Very Long
Subword (BPE)	Moderate	Excellent	Moderate

Step 2: Vocabulary and Token IDs

Once the text is broken into tokens, we must map each unique token to a unique integer, known as a Token ID. This mapping is stored in a dictionary called the Vocabulary [46].

Building the Vocabulary

To build a vocabulary from a corpus, we extract all unique tokens, sort them, and assign an index to each [47].

```
# Example of building a simple vocabulary
unique_tokens = sorted(list(set(preprocessed_text)))
vocab = {token: i for i, token in enumerate(unique_tokens)}
```

Special Context Tokens

In addition to standard text tokens, LLMs require special tokens to handle specific contexts:

- * `<|unk|>` **(Unknown)**: Used by word-based tokenizers for words not in the vocabulary. Note: BPE-based models like GPT-2 do not typically need this because they can fall back to individual characters [\[48\]](#).
- * `<|endoftext|>` **:** Acts as a marker signaling the end of one document and the start of another [\[49\]](#).
- * `<|pad|>` **(Padding)**: Used to ensure all sequences in a training batch have the same length [\[50\]](#).

Step 3: Creating Input-Target Pairs

LLMs are trained using a self-supervised learning objective: predicting the next token in a sequence [\[51\]](#).

The Sliding Window Approach

To generate training data, we use a sliding window over the tokenized text. Given a "Context Length," we create an input sequence (X) and a target sequence (Y). The target sequence is the input sequence shifted by one position [\[52\]](#).

Example with Context Length 4:

- * **Input (X):** `[token1, token2, token3, token4]`
- * **Target (Y):** `[token2, token3, token4, token5]`

In this single pair, there are multiple prediction tasks, such as predicting `token2` given `[token1]`, or predicting `token5` given the full context `[token1, token2, token3, token4]` [\[53\]](#).

DataLoaders and Batching

For efficient training, we group these pairs into batches. A `DataLoader` manages this process, utilizing two key parameters:

- * **Batch Size:** The number of input-target pairs processed before the model updates its parameters [\[54\]](#).
- * **Stride:** How many tokens the sliding window moves after each sample [\[55\]](#).

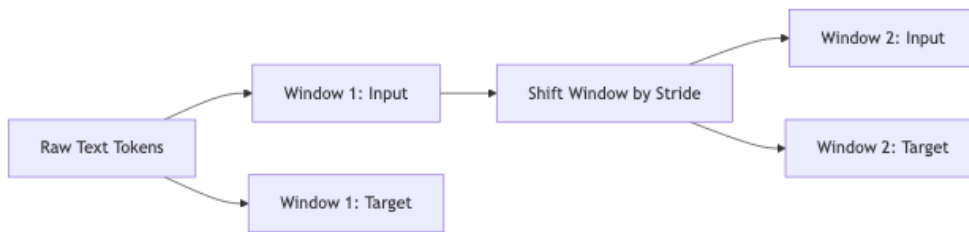


Figure 1: *Sliding Window Data Generation*

Step 4: Token Embeddings

After converting tokens to IDs, we transform those IDs into continuous vector representations [\[56\]](#).

Why Vectors?

Token IDs are arbitrary integers that do not convey semantic relationships. By representing words as vectors in a high-dimensional space, we can ensure that semantically similar words are placed closer together [\[57\]](#).

The Embedding Layer as a Lookup Table

In PyTorch, an embedding layer is essentially a weight matrix. When we pass a Token ID to this layer, it retrieves the corresponding row from the matrix [\[58\]](#).

```
import torch.nn as nn

# Example for GPT-2 scale
vocab_size = 50257
embedding_dim = 768
embedding_layer = nn.Embedding(vocab_size, embedding_dim)

# Input: Batch of 8 sequences, each 4 tokens long
input_ids = torch.randint(0, vocab_size, (8, 4))
token_embeddings = embedding_layer(input_ids)
# Output shape: (8, 4, 768)
```

Initially, these vectors are initialized with random values and are optimized during the training process [\[59\]](#).

Step 5: Positional Embeddings

The Transformer architecture processes all tokens in a sequence simultaneously and has no inherent sense of word order [\[60\]](#).

Absolute vs. Relative Positioning

To fix this, we must inject positional information into the embeddings. * **Absolute Positional Embeddings:** We create an embedding matrix where each position has its own unique vector. These vectors are added directly to the token embeddings [\[61\]](#). * **Relative Positional Embeddings:** These focus on the distance between tokens. While useful, GPT models typically use absolute embeddings [\[62\]](#).

Generating the Final Input

The final input to the LLM is the sum of the token embedding and the positional embedding [\[63\]](#).

```
# Context length of 4
context_length = 4
pos_embedding_layer = nn.Embedding(context_length, embedding_dim)

# Generate position indices [0, 1, 2, 3]
pos_indices = torch.arange(context_length)
pos_embeddings = pos_embedding_layer(pos_indices)

# Final input = Token Embeddings + Positional Embeddings
input_embeddings = token_embeddings + pos_embeddings
```

Summary of the Preprocessing Pipeline

The transformation from raw text to model input follows a linear pipeline: 1. **Tokenization:** Raw text is split into units using BPE [\[64\]](#). 2. **ID Mapping:** Tokens are converted into integers based on a vocabulary [\[65\]](#). 3. **Batching:** Data is organized into input-target pairs using a sliding window [\[66\]](#). 4. **Token Embedding:** Integer IDs are mapped to high-dimensional vectors [\[67\]](#). 5. **Positional Embedding:** Vectors representing the position of each token are added to the token vectors [\[68\]](#). 6. **Model Input:** The resulting "Input Embeddings" are fed into the Transformer blocks [\[69\]](#).

The Attention Mechanism

Context and Motivation

The evolution of Large Language Models (LLMs) is fundamentally a history of solving the "long-term dependency" problem. In the era preceding the Transformer architecture, sequence modeling relied heavily on Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. While these architectures introduced the concept of a "hidden state" to act as a memory cell, they suffered from a critical bottleneck: the encoder was forced to compress an entire input sequence into a single fixed-length vector [\[70\]](#).

As sequences grew in length, this "context loss" became debilitating. For a model to understand a sentence like *"The cat that was sitting on the mat which was next to the dog jumped,"* it must specifically link the subject ("cat") with the action ("jumped"), despite the intervening clauses about mats and dogs [\[71\]](#). In a standard RNN, the influence of the word "cat" often fades by the time the model processes "jumped."

The attention mechanism revolutionized this by allowing the model to selectively "attend" to any part of the input sequence at each step [\[72\]](#). Instead of relying on one final hidden state, the model gains a "Dynamic Focus," enabling it to access all previous hidden states and assign them different weights of importance [\[73\]](#). This chapter details the construction of this mechanism from its simplest mathematical form to the complex multi-head systems used in modern LLMs.

Simplified Self-Attention: The Purest Form

To understand the engine of a Transformer, we must first strip away trainable parameters and focus on the raw mathematical objective: converting static word embeddings into "enriched" context vectors [\[74\]](#).

From Embeddings to Context

A standard word embedding captures the general semantic meaning of a word. However, it is static; the embedding for "journey" does not contain information about how the word relates to other words in a specific sentence. The goal of attention is to transform these embeddings into context vectors that incorporate information from neighboring tokens [\[75\]](#).

The Role of the Dot Product

The fundamental operation used to determine the relationship between two vectors is the **dot product**. In a vector space, the dot product quantifies alignment: - If two vectors are parallel (aligned), their dot product is high. - If they are perpendicular (orthogonal), their dot product is zero. - If they point in opposite directions, it is negative.

In the context of LLMs, a high dot product between the vector for "Journey" and "Starts" suggests these words are semantically or syntactically related in the current sequence [\[76\]](#).

Implementation Step-by-Step

1. **Compute Attention Scores (w):** For a target word (the "query"), calculate the dot product between its embedding and every other word's embedding in the sequence.
2. **Normalization (Softmax):** These raw scores are transformed into weights that sum to 1.0. This makes the scores interpretable as percentages [\[77\]](#).
3. **Weighted Sum:** Multiply each original embedding by its corresponding attention weight and sum the results. This produces the context vector [\[78\]](#).

Table 3: Simplified Attention Example

Token	Embedding (x)	Score ($x_2 \cdot x_i$)	Weight (Softmax)
Your	[1.0, 0.5, 0.2]	0.45	0.13
Journey	[0.2, 1.2, 0.5]	1.49	0.24
Starts	[0.5, 1.0, 0.8]	1.40	0.23

Self-Attention with Trainable Weights

While the simplified version captures similarity, it lacks the ability to learn complex relationships that aren't purely based on embedding alignment. Modern LLMs introduce three trainable weight matrices: W_q (Query), W_k (Key), and W_v (Value) [\[79\]](#).

The Query, Key, and Value Analogy

- **Query (\$q\$):** What the current token is "looking for."
- **Key (\$k\$):** A description of what each token "offers."
- **Value (\$v\$):** The actual information content to be extracted once a match is found [\[80\]](#).

By multiplying the input embedding x by these matrices, we project the data into three distinct spaces: $q^{(i)} = x^{(i)}W_q$, $k^{(i)} = x^{(i)}W_k$, $v^{(i)} = x^{(i)}W_v$

Scaled Dot-Product Attention

A critical refinement is scaling the dot product. As the dimensionality (d_k) of the keys increases, the variance of the dot products grows, pushing the softmax function into regions with extremely small gradients [\[81\]](#). To counteract this, the scores are divided by $\sqrt{d_k}$: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ This scaling ensures the variance of the scores remains close to 1.0, leading to more stable training [\[82\]](#).

Causal Self-Attention

In generative tasks like next-word prediction, the model must not be allowed to "cheat" by looking at future tokens. This is solved via **Causal Attention** [\[83\]](#).

The Masking Mechanism

To prevent the model from accessing future information, we apply a mask to the attention scores before the softmax step.

1. **Identify Future Tokens:** Any position $j > i$ is a future token relative to position i .
2. **Apply Negative Infinity:** Replace these scores with $-\infty$.
3. **Softmax:** When $e^{-\infty}$ is calculated, it becomes 0. Consequently, the attention weights for all future tokens are zeroed out [\[84\]](#).

Dropout in Attention

To prevent overfitting, a Dropout layer is often applied to the attention weights. This randomly zeros out a percentage of the weights during training, forcing the model to find multiple pathways to represent the context [\[85\]](#).

Multi-Head Attention (MHA)

Multi-Head Attention allows the model to attend to information from different representation subspaces simultaneously [\[86\]](#).

The Wrapper Approach vs. Weight Splitting

There are two primary ways to implement MHA: 1. **The Wrapper Approach:** Create multiple independent causal attention modules and concatenate their outputs [\[87\]](#). 2. **Weight Splitting:** Use a single large weight matrix for Q, K, V , then "split" the resulting tensors into multiple heads. This is more computationally efficient [\[88\]](#).

Mathematical Flow of Weight Splitting

The process involves tensor reshaping to handle multiple heads: 1. **Linear Projection:** Project input X to Q, K, V using larger weight matrices. 2. **Reshape:** Split the output dimension into (Number of Heads, Head Dimension) [\[89\]](#). 3. **Transpose:** Swap the "Tokens" and "Heads" dimensions to allow the model to treat each head as a separate batch [\[90\]](#). 4. **Compute Attention:** Perform scaled dot-product attention across all heads in parallel. 5. **Concatenate and Project:** Merge the heads back into a single vector and pass through a final linear layer [\[91\]](#).

Implementation Detail: The Multi-Head Class

The following Python structure represents the optimized version of Multi-Head Attention using weight splitting and PyTorch's `nn.Linear` layers.

```
import torch
import torch.nn as nn

class MultiHeadAttention(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, num_heads):
        super().__init__()
        assert d_out % num_heads == 0, "d_out must be divisible by num_heads"

        self.d_out = d_out
        self.num_heads = num_heads
        self.head_dim = d_out // num_heads

        self.W_query = nn.Linear(d_in, d_out, bias=False)
        self.W_key = nn.Linear(d_in, d_out, bias=False)
        self.W_value = nn.Linear(d_in, d_out, bias=False)
```

```

self.out_proj = nn.Linear(d_out, d_out)
self.dropout = nn.Dropout(dropout)
self.register_buffer('mask', torch.tril(torch.ones(context_length, context_length)))

def forward(self, x):
    b, num_tokens, d_in = x.shape

    # Step 1: Linear projections
    keys = self.W_key(x) # (b, num_tokens, d_out)
    queries = self.W_query(x)
    values = self.W_value(x)

    # Step 2: Split into heads by reshaping and transposing
    keys = keys.view(b, num_tokens, self.num_heads, self.head_dim).transpose(1, 2)
    queries = queries.view(b, num_tokens, self.num_heads, self.head_dim).transpose(1, 2)
    values = values.view(b, num_tokens, self.num_heads, self.head_dim).transpose(1, 2)

    # Step 3: Scaled dot-product attention
    attn_scores = queries @ keys.transpose(2, 3)
    mask_bool = self.mask[:num_tokens, :num_tokens].bool()
    attn_scores.masked_fill_(~mask_bool, -float('inf'))

    attn_weights = torch.softmax(attn_scores / (keys.shape[-1]**0.5), dim=-1)
    attn_weights = self.dropout(attn_weights)

    # Step 4: Combine heads
    context_vec = (attn_weights @ values).transpose(1, 2).contiguous()
    context_vec = context_vec.view(b, num_tokens, self.d_out)

    return self.out_proj(context_vec)

```

Advanced Considerations

Registering Buffers

In the implementation above, the causal mask is registered as a "buffer" using `self.register_buffer`. Unlike parameters, buffers are not updated during backpropagation but are part of the model's state. This ensures the mask is automatically moved to the correct device (CPU or GPU) alongside the model weights [\[92\]](#).

Numerical Stability

The use of `torch.softmax` can lead to numerical instability if the input values are extremely large. PyTorch's internal implementation of softmax mitigates this by subtracting the maximum value in the input vector before exponentiation:
$$\text{softmax}(x_i) = \frac{e^{x_i - \max(x)}}{\sum e^{x_j - \max(x)}}$$
 This mathematical identity ensures that the largest value becomes $e^0 = 1$, preventing the exponential from blowing up [\[93\]](#).

Variance and Scaling

The choice of $\sqrt{d_k}$ as a scaling factor is not arbitrary. If the components of Q and K are independent random variables with mean 0 and variance 1, their dot product has mean 0 and variance d_k . By dividing by $\sqrt{d_k}$, we return the variance of the dot product to 1, which keeps the softmax in a range where it has meaningful gradients [\[94\]](#).

LLM Architecture & Components

Context and Overview of the LLM Architecture

The construction of a Large Language Model (LLM) is a multi-stage engineering endeavor that transitions from foundational data preparation to the assembly of complex neural architectures. Having previously established the mechanisms of tokenization, self-attention, and embeddings, the focus now shifts to the structural "skeleton" that houses these components: the Transformer architecture.

The architecture of a modern LLM, specifically those following the GPT (Generative Pre-trained Transformer) lineage, is a deep neural network designed for the task of next-token prediction. While the attention mechanism is often described as the "heart" of the Transformer, it is embedded within a "Transformer Block," a modular unit that is replicated numerous times to create a deep network. For example, the GPT-2 small model (124 million parameters) utilizes 12 of these blocks stacked sequentially [\[95\]](#).

The architectural journey involves three primary stages: 1. **Foundational Assembly:** Laying the foundations including tokenization and attention mechanisms. 2. **Pre-training:** Training the model on large datasets. 3. **Fine-tuning:** Adapting the pre-trained model for specific tasks [\[96\]](#).

This chapter details the assembly of the GPT-2 architecture, specifically the 124M parameter version. This model serves as a pedagogical gold standard because it is the smallest version of GPT-2, making its scale manageable for learning while still possessing the complexity of state-of-the-art systems [\[97\]](#).

The Transformer Block: A Deep Dive

The Transformer block is the fundamental building block of the LLM. It is designed to process input embeddings through a series of layers to produce an output that can be used to predict the next word in a sequence. This modular design allows for the stacking of multiple blocks—12 in the case of GPT-2 small—to form the complete architecture [\[98\]](#).

Structural Components

A single Transformer block consists of five primary sub-components: 1. **Masked Multi-Head Attention:** The mechanism that allows the model to focus on different parts of the input sequence. 2. **Layer Normalization (LayerNorm):** A technique to stabilize training. 3. **Dropout:** A regularization technique used to prevent overfitting. 4. **Feed-Forward Network (FFN):** A sub-module consisting of linear layers and an activation function. 5. **Shortcut Connections:** Also known as residual or skip connections, these allow gradients to flow through the network more easily during backpropagation [\[99\]](#).

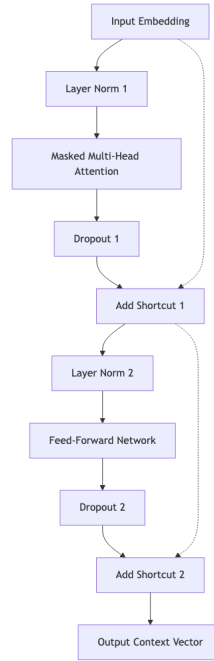


Figure 2: *Transformer Block Flow*

Layer Normalization (LayerNorm)

Layer normalization is essential for training deep networks. Without it, models are highly susceptible to the **vanishing gradient** or **exploding gradient** problems. In deep architectures, gradients can become progressively smaller as they propagate backward; if this happens, weight updates are not made, and learning becomes stagnant [\[100\]](#).

LayerNorm is applied at multiple places: within the Transformer block (before the attention and feed-forward modules) and again after the final Transformer block before the output processing steps [\[101\]](#).

The Mathematical Procedure: For an input vector x , LayerNorm performs the following: 1. **Calculate Mean (μ):** The average of the elements. 2. **Calculate Variance (σ^2):** The average squared deviation from the mean. 3. **Normalize:** $\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$, where ϵ is a small constant to prevent division by zero. 4. **Scale and Shift:** $y = \gamma \hat{x} + \beta$. Here, γ (scale) and β (shift) are trainable parameters that allow the model to learn the optimal scaling and shifting for the activations [\[102\]](#).

In the GPT-2 architecture, LayerNorm is applied *before* the attention and feed-forward modules. This placement is a key part of the Transformer block's organization [\[103\]](#).

GELU Activation and the Feed-Forward Network

The Feed-Forward Network (FFN) within the Transformer block is a sub-module that processes the data after the attention stage. It consists of linear layers and a non-linear activation function [\[104\]](#).

The Expansion-Contraction Design: The FFN typically follows an expansion-contraction pattern: 1. **Linear Expansion:** The input is projected into a higher-dimensional space. 2. **Activation Function:** A non-linear activation is applied. 3. **Linear Contraction:** The vector is projected back down to the original embedding dimension.

This structure is a standard component of the Transformer block [\[105\]](#).

The Gaussian Error Linear Unit (GELU): While many neural networks use ReLU (Rectified Linear Unit), GPT models utilize GELU (or GeLU). ReLU is defined as $f(x) = \max(0, x)$. GELU is a different activation function that is commonly used in modern Transformers to introduce non-linearity [\[106\]](#).

Shortcut Connections

Shortcut connections (or skip connections) add the input of a module to its output. Mathematically, this is vital for backpropagation. They were originally proposed to solve the **vanishing gradient** problem, where gradients become too small to effectively update weights. By providing a "shortcut" for the gradient to flow through, these connections ensure that learning remains effective even in very deep networks [\[107\]](#).

Assembling the Full GPT Architecture

The complete GPT model is an assembly of embedding layers, a stack of Transformer blocks, and a final output head.

The Configuration (GPT-2 124M)

Table 4: GPT-2 Small Configuration

Parameter	Value	Description
<code>vocab_size</code>	50,257	Number of unique tokens in the vocabulary
<code>context_length</code>	1,024	Max tokens the model can process at once
<code>emb_dim</code>	768	Dimensionality of token and positional embeddings
<code>n_heads</code>	12	Number of attention heads per block
<code>n_layers</code>	12	Number of Transformer blocks stacked
<code>drop_rate</code>	0.1	Dropout probability (10%)
<code>qkv_bias</code>	False	Whether to use bias in Query/Key/Value projections

The Forward Pass Workflow

The data flows through the model in a specific sequence [\[108\]](#):

1. **Token and Positional Embeddings:** Input token IDs are converted into token embeddings, and positional embeddings are added to them.
2. **Dropout:** A dropout layer is applied to the combined embeddings.
3. **Transformer Block Stack:** The data passes through 12 successive Transformer blocks.
4. **Final LayerNorm:** After the final Transformer block, a global layer normalization is applied.
5. **Linear Output Head:** The final vectors are projected to the vocabulary size to produce **Logits**, which are used to predict the next token.

Parameter Count and Weight Tying

The GPT-2 124M model uses a technique called **Weight Tying**, where the weights of the token embedding layer and the final linear output layer are shared. This reduces the total parameter count [\[109\]](#).

Implementation: The GPT Model Class

The implementation in PyTorch involves nesting the `TransformerBlock` and `LayerNorm` classes within a master `GPTModel` class.

```

import torch
import torch.nn as nn

class GPTModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate"])

        # Stack of Transformer Blocks
        self.trf_blocks = nn.Sequential(
            *[TransformerBlock(cfg) for _ in range(cfg["n_layers"])]
        )

        self.final_norm = LayerNorm(cfg["emb_dim"])
        self.out_head = nn.Linear(cfg["emb_dim"], cfg["vocab_size"], bias=False)

    def forward(self, in_idx):
        batch_size, seq_len = in_idx.shape
        tok_embeddings = self.tok_emb(in_idx)

        # Generate position IDs dynamically
        pos_embeddings = self.pos_emb(torch.arange(seq_len, device=in_idx.device))

        x = tok_embeddings + pos_embeddings
        x = self.drop_emb(x)
        x = self.trf_blocks(x)
        x = self.final_norm(x)
        logits = self.out_head(x)
        return logits

```

[Source: Video: "Coding the 124 million parameter GPT-2 model" @ 46:50]

Generating Text: From Logits to Tokens

The output of the `GPTModel` is a tensor that must be transformed back into tokens through an iterative process [\[110\]](#).

The Iterative Generation Loop

LLMs generate text one token at a time: 1. **Input:** A sequence of tokens. 2. **Forward Pass:** The model processes the sequence and produces logits. 3. **Token Selection:** A strategy is used to pick the next token ID from the logits. 4.

Append: The new token ID is appended to the input sequence. 5. **Repeat:** The process repeats to generate a full sequence.

Greedy Search vs. Sampling

In **Greedy Search**, the token with the highest logit value is selected. To introduce variety, the **Softmax** function can be applied to convert logits into probabilities, allowing the model to sample from the distribution [\[111\]](#).

Worked Example: Next Token Prediction

Assume an input "Every effort moves you" (4 tokens). 1. The model receives the tokens and produces an output matrix. 2. The last row of the output corresponds to the prediction for the token following "you". 3. The model identifies the index of the maximum value in that row and decodes it to get the next word, such as "forward" [\[112\]](#).

Summary

The LLM architecture is an orchestration of normalization, attention, and feed-forward layers. By stacking Transformer blocks, the model gains the capacity to represent complex linguistic relationships.

Key takeaways include: - **Transformer Blocks:** These are the fundamental units, repeated (12 times in GPT-2 small) to create the model. - **Stability Mechanisms:** LayerNorm and Shortcut connections are essential for training deep models and mitigating the vanishing gradient problem. - **Iterative Generation:** Text is produced token-by-token, with each new token being fed back into the model.

At this stage, the model has the structure to understand language but contains random weights. The next phase involves pre-training these parameters on massive text corpora [\[113\]](#).

Pre-training & Model Evaluation

Context and Overview

Building a Large Language Model (LLM) from scratch involves a transition from architectural design to the iterative process of pre-training. While the Transformer

block provides the structural foundation, the intelligence of the model is derived from its parameters, which must be optimized to predict the next token in a sequence. This chapter focuses on the second stage of LLM development: Pre-training and Model Evaluation.

In traditional machine learning, loss functions like Mean Squared Error (MSE) for regression or Cross-Entropy for classification are well-defined. For LLMs, the task is framed as a next-token prediction problem, requiring a quantitative metric to transform qualitative intuition—such as knowing the output does not look good—into a mathematical objective that can be minimized using gradient descent [\[114\]](#). We will explore the mechanics of measuring loss, the implementation of a pre-training loop, and decoding strategies like temperature scaling and top-k sampling to reduce randomness in model output.

Measuring the LLM Loss Function

The primary objective of pre-training is to minimize the difference between the model's predicted token and the actual next token in the training data. To achieve this, we must first understand the relationship between inputs, targets, and logits.

Inputs and Targets

In the context of an LLM, the "Target" refers to the true values the model is expected to approximate. Because LLMs are auto-regressive, the targets are derived directly from the input text by shifting the sequence to the right by one position [\[115\]](#).

Consider a batch size of 2 with a context length of 4. If the first batch input is "every effort moves you" and the second is "I really like it", the input tensor contains the token IDs for these words. The target tensor contains the IDs for the words that follow each token in the input:

- For the input "every", the target is "effort".
- For the input "every effort", the target is "moves".
- For the input "every effort moves", the target is "you".

This results in distinct prediction tasks per sample in the batch. Consequently, the target tensor matches the input dimensions [\[116\]](#).

From Logits to Probabilities

The GPT architecture outputs a "logits" tensor. These raw values are unnormalized and do not represent probabilities. To convert these into a probability distribution, we apply the Softmax function. This ensures that for every token position, the sum of probabilities across the entire vocabulary equals 1.0. During evaluation, a naive method is to use `torch.argmax` to select the index with the highest probability, a method known as greedy decoding [\[117\]](#).

Cross-Entropy Loss and Negative Log-Likelihood

To quantify performance, we use Cross-Entropy loss. Mathematically, this involves identifying the probability the model assigned to the *correct* target token. If the target token ID for the first position is 3626 ("effort"), we look at the probability at index 3626 in the model's output distribution [\[118\]](#).

The goal is to drive these specific probabilities as close to 1.0 as possible. In PyTorch, this is implemented using `torch.nn.functional.cross_entropy(logits.view(-1, vocab_size), targets.view(-1))`. This function automatically applies Softmax and calculates the loss in a single step [\[119\]](#).

Evaluating Performance on Real Datasets

To move beyond toy examples, we utilize a real-world dataset, such as the short story "The Verdict" by Edith Wharton (1906). This text contains approximately 20,000 characters, which translates to roughly 5,000 tokens when processed through a Byte Pair Encoding (BPE) tokenizer [\[120\]](#).

Data Splitting and Loading

Standard practices apply: we split the data into a training set (90%) and a validation set (10%). The training loss indicates how well the model is learning the patterns in the data, while the validation loss measures its performance on unseen text [\[121\]](#).

We implement a data loader to create input-target pairs. A critical parameter here is the **stride**. If the context length is 256, the stride determines the step size for moving the context window, which can be set to avoid overlap or to create specific sampling patterns [\[122\]](#).

Table 5: *Data Loader Configuration*

Parameter	Value (Example)	Description
Batch Size	2	Number of sequences processed simultaneously.
Context Length	256	Maximum tokens the model "sees" per prediction.
Stride	256	Step size for moving the context window.
Vocabulary Size	50,257	Total unique tokens in the BPE tokenizer.

The Evaluation Loop

Before training begins, we establish a baseline. By passing the training and validation loaders through the uninitialized model, we calculate the initial loss. For a large vocabulary, the initial cross-entropy loss is typically high, reflecting the model's total uncertainty [\[123\]](#).

The Pre-training Loop Implementation

Pre-training is the process of updating the model's parameters to minimize the loss.

Parameter Scale

The parameters in a standard small GPT model are distributed across several components:

1. **Embeddings:** Token and positional embeddings.
2. **Transformer Blocks:** Multiple blocks containing multi-head attention and feed-forward networks.
3. **Output Head:** The final linear layer projects the hidden state back to the vocabulary size.

The Training Step

The training loop follows a standard pattern:

1. **Forward Pass:** Compute logits for a batch of input tokens.
2. **Loss Calculation:** Compare logits against target tokens using cross-entropy.
3. **Backward Pass:** Execute backpropagation to calculate the gradient of the loss with respect to parameters.
4. **Weight Update:** Use an optimizer, typically **AdamW**, to update weights. AdamW is preferred for better convergence [\[124\]](#).

Monitoring and Overfitting

During training, we track the loss over epochs. A common observation in small-scale pre-training is that the training loss drops significantly while the validation loss remains high. This divergence is a sign of **overfitting**, where the model memorizes the training text rather than learning the underlying language structure [\[125\]](#).

Advanced Decoding Strategies

Greedy decoding often leads to repetitive or random-looking text. To introduce variety, we employ probabilistic sampling.

Temperature Scaling

Temperature (T) is a hyperparameter applied to the logits before the Softmax layer:

- $T = 1.0$: The distribution remains unchanged.
- $T < 1.0$ (**Low Temperature**): The distribution becomes "sharper," making the model more confident.
- $T > 1.0$ (**High Temperature**): The distribution becomes "flatter," increasing diversity but also the risk of nonsensical output [\[126\]](#).

Top-k Sampling

Top-k sampling restricts the model's choices to the K most likely tokens. The process involves:

1. Identifying the top K logits.
2. Setting all other logits to a very low value (negative infinity).
3. Applying Softmax and sampling from the remaining K values.

This ensures the model only samples from a pool of sensible candidates [\[127\]](#).

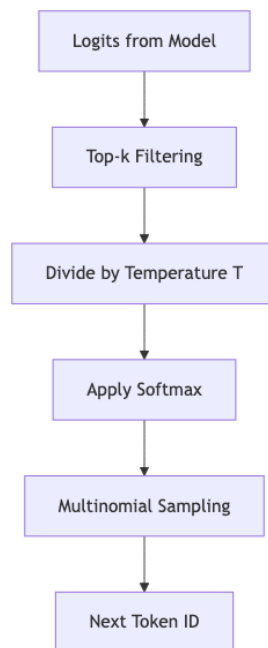


Figure 3:

Loading Pre-trained Weights from OpenAI

To achieve high-quality output, we can load weights from OpenAI's publicly released GPT-2 models, which were trained on massive datasets [\[128\]](#).

Downloading and Extracting Parameters

OpenAI's weights include several files:

- `model.ckpt` : The actual weight values.
- `encoder.json` : The vocabulary mapping.
- `hparams.json` : Model configuration (e.g., number of layers and embedding dimensions) [\[129\]](#).

Weight Mapping and Assignment

We must align the checkpoint variables with our model's internal structure. This involves mapping token embeddings, positional embeddings, and the parameters within the Transformer blocks [\[130\]](#).

A notable detail in GPT-2 is **weight tying**, where the weights used for the token embedding layer are recycled for the final output linear head [\[131\]](#).

Saving and Loading Model States

For long-running training sessions, it is essential to save the model's state. In PyTorch, this is handled via the `state_dict`, a dictionary that maps each layer to its parameter tensor [\[132\]](#).

Saving the Model and Optimizer

Saving only the model weights is often insufficient for resuming training. Optimizers like AdamW maintain states such as running averages of gradients. If these are not saved, the optimizer loses its momentum upon restart [\[133\]](#).

Loading the Checkpoint

To resume, we must instantiate the model and optimizer, then load the state dictionaries using `load_state_dict` [\[134\]](#).

Summary

This chapter has detailed the process of taking a Transformer architecture from a randomly initialized state to a functional language generator. We established that the core of LLM training is the minimization of Cross-Entropy loss. We explored the practicalities of data loading using "The Verdict" dataset and implemented a pre-training loop using the AdamW optimizer.

To address the limitations of greedy decoding, we introduced temperature scaling and top-k sampling. Finally, we discussed loading OpenAI's gpt2-small weights into our custom-built architecture to enable the generation of coherent text.

Classification Fine-tuning

Introduction to Classification Fine-tuning

Building a large language model (LLM) from scratch involves three primary stages. Stage one focuses on the fundamental architecture, including attention mechanisms and data preparation. Stage two involves pre-training the model on massive datasets to learn general language patterns and loading those pre-trained weights [\[135\]](#). However, a pre-trained model, while capable of generating coherent text, is often insufficient for specific industrial or academic applications. This leads to stage three: fine-tuning.

Fine-tuning is the process of adapting a pre-trained model to a specific task by training it further on a smaller, domain-specific dataset [\[136\]](#). By feeding the model additional data, we optimize its weights and biases to perform a targeted function.

There are two broad categories of fine-tuning: 1. **Instruction Fine-tuning:** Training the model to follow specific prompts, such as answering questions or following commands. 2. **Classification Fine-tuning:** Adapting the model to categorize input text into predefined buckets, such as "Spam" vs. "Not Spam" [\[137\]](#). Unlike instruction fine-tuning, the model is architecturally modified to output a class label.

This chapter focuses on the end-to-end implementation of classification fine-tuning, using a real-world SMS spam detection dataset to transform a general-purpose GPT model into a specialized classifier.

Data Preparation and Pre-processing

The quality of a fine-tuned model is directly dependent on the data used to adapt it. For this project, we utilize the SMS Spam Collection dataset from the UCI Irvine Machine Learning Repository [\[138\]](#).

Dataset Analysis and Balancing

The raw dataset contains 4,825 "ham" (legitimate) messages and only 747 "spam" messages [\[139\]](#). If trained on this imbalanced data, the model might achieve high accuracy simply by predicting "ham" every time.

To mitigate this, we perform subsampling to create a balanced dataset. We randomly select 747 ham messages to match the 747 spam messages, resulting in a

balanced set of 1,494 instances [\[140\]](#). We then encode these labels numerically: 0 for ham and 1 for spam.

Data Splitting

Following standard machine learning practices, the balanced data is split into three subsets: * **Training (70%)**: Used to update the model weights. * **Validation (10%)**: Used to monitor performance during training. * **Testing (20%)**: A final, "unseen" set used to evaluate the model's generalization capability [\[141\]](#).

Table 6: Data Split Distribution

Subset	Percentage	Number of Samples
Training	70%	1,045
Validation	10%	149
Testing	20%	300
Total	100%	1,494

Tokenization and Data Loading

LLMs do not process raw text; they process token IDs. For this implementation, we use the Byte Pair Encoding (BPE) tokenizer used by GPT-2 [\[142\]](#).

The Padding Challenge

In a batch-processing environment, every input sequence in a batch must have the same length to form a uniform tensor. SMS messages vary significantly in length. We adopt a padding approach where we identify the longest sequence in our dataset (which is 120 tokens) and pad all shorter sequences using the `<|endoftext|>` token, which corresponds to ID `50256` in the GPT-2 vocabulary [\[143\]](#).

Implementing the PyTorch Dataset

We define a custom `SpamDataset` class to handle tokenization and padding. This class ensures that every call to `__getitem__` returns a tensor of exactly 120 token IDs and its corresponding label (0 or 1) [\[144\]](#).

```
# Conceptual implementation of the Dataset
class SpamDataset(Dataset):
    def __init__(self, csv_file, tokenizer, max_length=None, pad_token_id=50256):
        self.data = pd.read_csv(csv_file)
        self.encoded_texts = [tokenizer.encode(text) for text in self.data["text"]]

        if max_length is None:
            self.max_length = max([len(t) for t in self.encoded_texts])
        else:
            self.max_length = max_length

        # Pad or truncate to max_length
        self.encoded_texts = [
            t[:self.max_length] + [pad_token_id] * (self.max_length - len(t))
            for t in self.encoded_texts
        ]
```

DataLoaders

The `DataLoader` class in PyTorch wraps the dataset and provides batches of data. For this project, we use a batch size of 8. This means the input tensor for each batch has a shape of `(8, 120)`, representing 8 messages each containing 120 tokens [\[145\]](#).

Architectural Modifications for Classification

To convert a generative model into a classifier, we must modify its "head"—the final layers that produce output.

The Classification Head

A standard GPT-2 model has an output layer that maps the hidden dimension (768 for GPT-2 small) to the vocabulary size (50,257). For classification, we replace this with a new linear layer that maps 768 hidden units to the number of classes (2 in our case) [\[146\]](#).

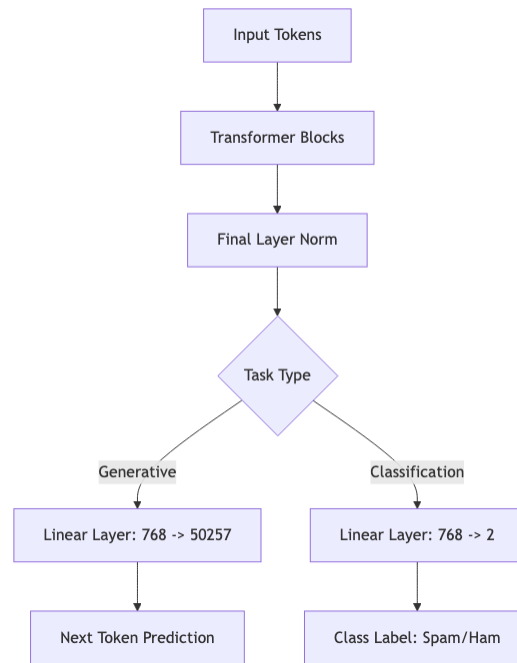


Figure 4: *Architectural Shift*

Freezing vs. Fine-tuning

Fine-tuning does not always require updating every parameter in the model. To save computational resources, we can freeze the majority of the model [\[147\]](#).

We specifically set `requires_grad = True` for only three components: 1. **The new Classification Head.** 2. **The 12th (Final) Transformer Block.** 3. **The Final Layer Norm** [\[148\]](#).

Extracting the Last Token

In a generative model, every input token produces an output. For classification, we use the output corresponding to the **last token**. Because GPT-2 uses causal self-attention, the last token is the only one that has integrated information from all preceding tokens in the sequence [\[149\]](#).

The Training Loop and Loss Function

Cross-Entropy Loss

For classification, we use Cross-Entropy Loss. This function measures the difference between the model's predicted probability distribution and the actual label [\[150\]](#).

Optimization with AdamW

We use the `AdamW` optimizer. During each step of the training loop: 1. A batch of 8 messages is passed through the model. 2. The output of the last token is extracted. 3. The loss is calculated against the true labels. 4. `loss.backward()` computes the gradients for the trainable parameters. 5. `optimizer.step()` updates the weights [\[151\]](#).

Evaluation Metrics

During training, we monitor loss and accuracy. Initially, a pre-trained model without fine-tuning performs poorly on this task—often around 46% accuracy—because the new classification head is randomly initialized [\[152\]](#). After fine-tuning for 5 epochs, the training accuracy typically reaches near 100%, with validation and test accuracies settling around 95-97% [\[153\]](#).

Model Inference and Deployment

The Inference Pipeline

To classify a new message, the following steps occur: 1. **Tokenization:** The text is converted to token IDs. 2. **Padding:** The sequence is padded to 120 tokens. 3. **Forward Pass:** The tensor is passed through the fine-tuned model. 4. **Logit Extraction:** The two output values (logits) for the last token are extracted. 5. **Argmax:** The index with the highest value is chosen as the predicted class [\[154\]](#).

Saving and Loading

To avoid retraining, we use `torch.save(model.state_dict(), "path.pth")` to save the optimized weights. These can be reloaded later using `model.load_state_dict(torch.load("path.pth"))` [\[155\]](#).

Summary

Classification fine-tuning is a powerful method for repurposing large-scale pre-trained models. Key takeaways include: * **Data Balancing:** Essential for preventing bias in classifiers. * **Padding and Tokenization:** Critical for preparing text for batch processing. * **Last-Token Significance:** The causal

nature of GPT models means the final token contains the most comprehensive context for classification. * **Parameter Efficiency:** Fine-tuning only the final layers provides a balance between performance and computational efficiency.

This foundation serves as a prerequisite for more complex adaptation techniques, such as instruction fine-tuning and Parameter-Efficient Fine-Tuning (PEFT) methods like LoRA [\[156\]](#).

Instruction Fine-tuning & Conclusion

Opening Context

The journey of building a Large Language Model (LLM) from scratch follows a rigorous progression: from data preparation and the implementation of the attention mechanism to the construction of the Transformer architecture and initial pre-training. However, a model that has only undergone pre-training—while statistically proficient at predicting the next token—often fails to function as a useful assistant. If asked to "fix the grammar" or "convert this sentence to passive voice," a base pre-trained model might simply continue the text by providing more examples of grammar errors or repeating the input rather than executing the command [\[157\]](#).

Instruction fine-tuning is the critical bridge between a raw, text-completing engine and a functional AI assistant. This chapter explores the methodology of supervised instruction fine-tuning, the complexities of data batching with variable sequence lengths, and the emerging frameworks for evaluating generative outputs. By the conclusion of this chapter, the foundational GPT model developed in previous sections will be transformed into a personal assistant capable of following natural language directives [\[158\]](#).

1. The Necessity of Instruction Fine-tuning

Pre-trained models, such as the GPT-2 weights provided by OpenAI, are trained on massive corpora to minimize cross-entropy loss on the next-token prediction task. While this imparts a "knowledgeable state" regarding language syntax and world facts, it does not align the model with human intent [\[157\]](#).

1.1 Bridging the Gap to Personal Assistants

Without fine-tuning, an LLM struggles with specific tasks like:

- * **Grammar Correction:** Instead of fixing a sentence, it might generate a similar incorrect sentence or repeat the prompt.
- * **Voice Conversion:** It may fail to understand the structural transformation required to move from active to passive voice [\[157\]](#).
- * **Domain Specificity:** In corporate environments, a base model lacks knowledge of private company policies, such as specific refund or return procedures [\[159\]](#).

Instruction fine-tuning allows a company to provide domain-specific instructions, such as "If a user asks about the return policy, provide these specific steps." This ensures the model is not just a general language engine but a relevant tool for specific use cases like e-commerce support [\[158\]](#).

1.2 Supervised Instruction Fine-tuning (SFT)

The approach used here is **Supervised Instruction Fine-tuning**. It is "supervised" because the model is provided with explicit pairs of instructions and desired responses. A typical training entry consists of three components:

1. **Instruction:** The task description (e.g., "Translate to French").
2. **Input:** The context for the task (e.g., "Where is the restaurant?").
3. **Output:** The ground-truth response (e.g., "Où est le restaurant?").

In cases where the instruction is self-contained (e.g., "What is the capital of Indonesia?"), the input field may be left blank [\[160\]](#).

2. Dataset Preparation and Prompt Formatting

For this implementation, we utilize a dataset of 1,100 instruction-response pairs. While smaller than industrial datasets like the Stanford Alpaca (52,000 pairs), it is sufficient to demonstrate the mechanics of fine-tuning [\[161\]](#).

2.1 The Alpaca Prompt Style

Researchers have found that the way a prompt is presented during training significantly impacts performance. The **Stanford Alpaca format** wraps the raw instruction and input into a structured template:

```
Below is an instruction that describes a task, paired with an input that provides fu
Write a response that appropriately completes the request.
```

```
#
## Instruction:
{instruction}

#
## Input:
{input}

#
## Response:
{output}
```

This structure helps the model distinguish between the meta-instruction, the specific task, and the expected response [\[162\]](#).

3. Advanced Data Batching Mechanics

Batching for instruction fine-tuning is more complex than standard pre-training. In instruction fine-tuning, every prompt-response pair has a different length, yet PyTorch tensors require uniform dimensions within a batch [\[163\]](#).

3.1 The Five-Step Batching Workflow

To organize data into training batches, we follow a sequential process: 1. **Formatting:** Apply the Alpaca template to the raw data. 2. **Tokenization:** Convert the formatted text into token IDs. 3. **Padding:** Ensure all sequences in a batch match the length of the longest sequence using a padding token. 4. **Target Creation:** Create target IDs by shifting the input IDs. 5. **Masking (Ignore Index):** Replace padding tokens in the target with `-100` so they are ignored by the loss function [\[164\]](#).

3.2 The Logic of the "Ignore Index"

In PyTorch, the `CrossEntropyLoss` function can use an `ignore_index` of `-100`. By replacing padding tokens with `-100`, we ensure the model is not penalized for failing to predict padding. We retain the end-of-text token because the model must learn when to stop generating [\[165\]](#).

4. Implementing DataLoaders

DataLoaders wrap the dataset in an iterable, allowing for efficient access to batches during training. This is vital for models with millions of parameters to improve efficiency [\[166\]](#).

4.1 Custom Collate Function

Because our sequences vary in length, we use a **custom collate function**. This function is called every time a batch is drawn to perform padding and masking dynamically for that specific batch [\[167\]](#).

5. Loading Pre-trained Weights

Rather than initializing the model with random weights, we load the pre-trained weights of GPT-2. For instruction fine-tuning, the 355-million parameter model is used to provide sufficient capacity for following instructions [\[168\]](#).

5.1 Model Configuration

The GPT-2 355M configuration includes: * **Parameters:** 355 Million. * **Transformer Blocks:** 24. * **Attention Heads:** 16. * **Embedding Dimension:** 1024 [\[169\]](#).

5.2 The "Knowledgeable State"

Loading these weights ensures the model already understands basic language syntax. Fine-tuning then acts as a refinement process, shifting the weights so the model learns to follow instructions effectively [\[170\]](#).

6. The Fine-tuning Training Loop

The training loop for fine-tuning operates on the instruction-response pairs to optimize the model weights.

6.1 Optimization Strategy

Key hyperparameters for the training process include: * **Epochs:** Typically 1 to 2, as more epochs on small datasets can lead to overfitting [\[171\]](#).

6.2 Loss Convergence

During training, we monitor the training and validation loss. A successful run shows the model learning to adopt the prompt structure and the nuances of the tasks [\[172\]](#).

7. Evaluating Generative Performance

Evaluating an instruction-tuned model is difficult because multiple different responses can be correct, even if they don't match the ground truth exactly [\[173\]](#).

7.1 Frameworks for Evaluation

1. **MMLU (Massive Multitask Language Understanding):** A benchmark that tests the breadth of the model's knowledge across various tasks [\[174\]](#).
2. **Human Preference:** Having humans rank model outputs, which is considered a gold standard [\[175\]](#).
3. **LLM-as-a-Judge:** Using a more powerful model to grade the responses of the smaller model [\[176\]](#).

8. Conclusion and Summary of the LLM Build

Building a Large Language Model from scratch transitions from raw data to a functional assistant.

8.1 The Journey Recap

- **Stage 1: Foundation.** Implementing tokenization, embeddings, and the attention engine [\[177\]](#).
- **Stage 2: Architecture.** Assembling Transformer blocks, including LayerNorm and feed-forward networks [\[178\]](#).
- **Stage 3: Fine-tuning.** Moving from next-token prediction to instruction-following using supervised datasets and evaluation metrics [\[179\]](#).

By understanding the "nuts and bolts" of these models, you transition from being a consumer of AI to an engineer capable of building and refining intelligent systems [\[180\]](#).

Conclusion

By completing this journey, you have transitioned from a conceptual understanding of artificial intelligence to a rigorous, architectural mastery of Large Language Models. You began by deconstructing the raw building blocks of language through tokenization and data preprocessing, eventually breathing life into those numbers through the self-attention mechanism. By assembling these components into a cohesive transformer architecture and navigating the complexities of pre-training and evaluation, you have gained the rare perspective of seeing the "ghost in the machine" not as magic, but as a sophisticated orchestration of linear algebra and probability.

The transition from pre-training to fine-tuning represents the final bridge between raw computational power and practical utility. You have learned how to steer a general-purpose model toward specific goals, whether through the precision of classification or the nuanced conversational capabilities of instruction tuning. This foundational knowledge ensures that you are no longer just a consumer of AI, but a creator capable of auditing, optimizing, and innovating within the field. You now possess the technical intuition required to understand not just how these models work today, but how they will evolve in the years to come.

As you look toward the horizon, the path forward is rich with specialized domains to explore. You might delve into parameter-efficient fine-tuning techniques like LoRA to democratize model training, or investigate Retrieval-Augmented Generation (RAG) to ground your models in real-world facts. Others may choose to focus on the ethics of alignment, the efficiency of model quantization, or the frontier of multimodal architectures. Regardless of which direction you choose, the first-principles approach you have cultivated here serves as your compass. The era of large language models is still in its infancy, and you are now uniquely equipped to help shape its future.

Glossary

Absolute Positional Embeddings — A technique where a unique vector is created for each specific position in a sequence and added to token embeddings to provide the model with a sense of word order.

Artificial Intelligence (AI) — The broadest category of technology encompassing any machine or system designed to mimic human intelligence.

Base Model — A model produced during the pre-training stage that has learned the fundamental structures of language from a massive, diverse dataset.

Batch Size — The specific number of input-target pairs that a model processes before it updates its internal parameters.

Byte Pair Encoding (BPE) — A subword tokenization scheme that represents frequent words as single units and breaks rare words into smaller subword components.

Character-Based Tokenization — A method of breaking text down where every individual character is treated as a separate token.

Classification Fine-tuning — The process of training a model to categorize data into specific labels, such as distinguishing between spam and non-spam.

Decoder — The component of the original Transformer architecture responsible for generating output text.

Deep Learning (DL) — A subset of machine learning that specifically utilizes deep neural networks to power tasks like image recognition.

Emergent Behavior — The phenomenon where a model develops the ability to perform complex tasks or reasoning that it was never explicitly programmed to do once it reaches a certain scale.

Encoder — The component of the original Transformer architecture designed to process input text.

Few-Shot Learning — A learning paradigm where a model is provided with several examples to establish a pattern for a task it wasn't explicitly trained for.

Fine-tuning — The second stage of building an LLM where a pre-trained model is refined by training it on a narrower, task-specific dataset.

Generative AI (GenAI) — A broad category of artificial intelligence focused on models that can generate various forms of content, including text, images, video, and audio.

Instruction Fine-tuning — A refinement process that trains a model to follow specific commands, effectively turning a base model into a helpful assistant.

Large Language Model (LLM) — A specialized application of deep learning designed as a neural network to understand, generate, and respond to human-like linguistic sequences.

Machine Learning (ML) — A subset of artificial intelligence characterized by machines that learn and adapt through interactions with data.

Neural network — A mathematical architecture inspired by the human brain's biological circuitry consisting of layers of computational units called neurons.

Neurons — Computational units within a neural network that process input data through weighted connections.

One-Shot Learning — A paradigm where a model is given a task description and exactly one example to learn how to perform a new task.

Parameters — The internal variables that a model adjusts during the training process to learn from data.

Positional Encoding — The process of adding a signal to tokens to indicate their specific position within a sequence.

Pre-training — The foundational phase of building an LLM where a model is trained on a massive corpus of unlabeled text to learn language structures.

Relative Positional Embeddings — A method of providing word order information that focuses on the relative distance between tokens rather than their absolute location.

Self-attention mechanism — A component of the Transformer architecture that allows a model to weigh the importance of different words in a sentence relative to each other regardless of distance.

Stride — The number of tokens a sliding window moves forward after each sample is taken during data generation.

Token ID — A unique integer assigned to each unique token in a model's vocabulary.

Tokenization — The process of breaking down raw text into smaller units such as words, subwords, or characters.

Transformer — A neural network architecture introduced in 2017 that uses self-attention mechanisms to capture long-range dependencies in text.

Unlabeled text — Data used in unsupervised learning where the model learns to predict the next word without explicit human-provided answers.

Vector Embedding — The process of mapping tokens into a high-dimensional mathematical space so that semantically similar words are positioned closer together.

Vocabulary — A dictionary that stores the mapping of unique tokens to their corresponding unique integers or Token IDs.

Word-Based Tokenization — A simple tokenization method that involves splitting text based on whitespace and punctuation.

Zero-Shot Learning — A capability where a model performs a task it wasn't explicitly trained for based only on a task description without any examples.

References & Resources

Introduction & Fundamentals

1. <https://openai.com/index/language-unsupervised/>
2. <https://arxiv.org/pdf/2005.14165>
3. https://www.business-standard.com/world-news/jpmorgan-chase-unveils-ai-powered-llm-suite-may-replace-research-analysts-124072600460_1.html
4. <https://openai.com/index/introducing-improvements-to-the-fine-tuning-api-and-expanding-our-custom-models-program/>

Data Preprocessing & Tokenization

1. https://pytorch.org/tutorials/beginner/basics/data_tutorial.html
2. https://colab.research.google.com/drive/1ceJDQUHBasbaTB74vg2Fz2N_O2h6C3T?usp=sharing
3. https://drive.google.com/file/d/1ukW7sNpd_4UxI6wKx27Q4fjX3pe6vzta/view?usp=sharing
4. <https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html>
5. <https://github.com/openai/tiktoken>
6. https://drive.google.com/file/d/1tF5aWLcZITuWy-1swK3_-63H2NCiYFWH/view?usp=sharing
7. <https://pytorch.org/docs/stable/data.html>
8. https://drive.google.com/file/d/1WxQo6_pG2wwBzxXErgBBPGG2vNt7XHUV/view?usp=sharing
9. https://github.com/rasbt/LLMs-from-scratch/blob/main/ch02/01_main-chapter-code/the-verdict.txt
10. <https://drive.google.com/file/d/1srG6CyModWArPVAumNIJCFDskFrC1CON/view?usp=sharing>
11. https://drive.google.com/file/d/1_lgiXKLwsQEgot4ziei9cWf9Zl91HXcH/view?usp=sharing
12. https://drive.google.com/file/d/1XKi9vE4_nr3B157LjgSx4iu6qUoOSgmB/view?usp=sharing

13. <https://docs.python.org/3/library/re.html>

The Attention Mechanism

1. <https://arxiv.org/pdf/1409.0473>
2. <https://pytorch.org/docs/stable/generated/torch.nn.Linear.html>
3. https://drive.google.com/file/d/1MJAE5wgH76oNzb8W_687ViYyUc7stNWN/view?usp=sharing
4. <https://pytorch.org/docs/stable/generated/torch.nn.Dropout.html>
5. <https://jalamar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>
6. <https://pytorch.org/docs/stable/generated/torch.nn.Module.html>
7. <https://drive.google.com/file/d/1b5b2PG55PjSYWkPhHy4Q3mQwjBXmCOTX/view?usp=sharing>
8. https://drive.google.com/file/d/1l2RXY24kWhQ8ur3WXLjnQD2n9_C6Fj42/view?usp=sharing
9. <https://arxiv.org/pdf/1706.03762>
10. <https://pytorch.org/docs/stable/generated/torch.tril.html>
11. <https://pytorch.org/docs/stable/generated/torch.triu.html>
12. <https://drive.google.com/file/d/17-GbtuioC4fnn9AhLR2m4oSgcpTNzIbu/view?usp=sharing>
13. <https://drive.google.com/file/d/13oYPEzurWAV4AbnFV9JWAG4uJVM65ZWB/view?usp=sharing>
14. https://pytorch.org/docs/stable/generated/torch.Tensor.masked_fill.html#torch.Tensor.masked_fill
15. <https://discuss.pytorch.org/t/what-is-the-difference-between-register-buffer-and-register-parameter-of-nn-module/32723>
16. <https://pytorch.org/docs/stable/generated/torch.nn.Softmax.html>

LLM Architecture & Components

1. <https://drive.google.com/file/d/1k4TwMW6HHDiS1tcbGO5ip3D3BonwSao/view?usp=sharing>

Pre-training & Model Evaluation

1. https://drive.google.com/file/d/1H6E79zXNsdG_Hn8y2wh6p6mj8o5GGJmD/view?usp=sharing

2. https://drive.google.com/file/d/1JX_DNJ9Xvty_IteiHhl_3TyOfahkGvvl/view?usp=sharing
3. <https://www.kaggle.com/datasets/xhlulu/openai-gpt2-weights>
4. https://pytorch.org/tutorials/beginner/basics/data_tutorial.html
5. https://pytorch.org/docs/stable/generated/torch.nn.functional.cross_entropy.html
6. https://pytorch.org/tutorials/recipes/recipes/what_is_state_dict.html#what-is-a-state-dict-in-pytorch
7. <https://drive.google.com/file/d/1mHo2cCTTo575ecaNnaDGAm2TTXvpXU3t/view?usp=sharing>
8. https://pytorch.org/docs/stable/generated/torch.optim.Optimizer.state_dict.html
9. <https://drive.google.com/file/d/1p87nemE6wFjGhkX4kC7KfGv1juMZJrMr/view?usp=sharing>
10. <https://pytorch.org/docs/stable/generated/torch.optim.AdamW.html>
11. <https://drive.google.com/file/d/1U03f--JS0He7zbUqcG5qHTBcZMa5YlM3/view?usp=sharing>
12. <https://github.com/openai/tiktoken>
13. <https://medium.com/@harshit158/softmax-temperature-5492e4007f71>
14. https://github.com/rasbt/LLMs-from-scratch/blob/main/ch02/01_main-chapter-code/the-verdict.txt
15. <https://pytorch.org/docs/stable/generated/torch.multinomial.html>
16. https://pytorch.org/tutorials/beginner/saving_loading_models.html
17. <https://drive.google.com/file/d/1k--iNpyoynmiHCyn4YC8yiZFbzDNT-nO/view?usp=sharing>
18. <https://pytorch.org/docs/stable/generated/torch.topk.html>

Classification Fine-tuning

1. https://pytorch.org/tutorials/beginner/basics/data_tutorial.html
2. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html
3. https://pytorch.org/docs/stable/generated/torch.nn.functional.cross_entropy.html
4. <https://archive.ics.uci.edu/dataset/228/sms+spam+collection>
5. <https://drive.google.com/file/d/1oXbvOT9t74pEXejQ3o7w9ZABGLjHmFsu/view?usp=sharing>
6. <https://pytorch.org/docs/stable/generated/torch.optim.AdamW.html>

7. <https://pytorch.org/docs/stable/generated/torch.save.html>
8. <https://github.com/openai/tiktoken>
9. <https://drive.google.com/file/d/1XcmKqTul6id86nUUvFsCgy-uBWttvWXf/view?usp=sharing>

Instruction Fine-tuning & Conclusion

1. <https://drive.google.com/file/d/1qyydJT5PCoeK4BopXlbJ3N1f3gPKA6i6/view?usp=sharing>
2. https://drive.google.com/file/d/1FQIzAGdBVgSqR_39Qx4jQYoAOYARkkld/view?usp=sharing
3. https://pytorch.org/tutorials/beginner/basics/data_tutorial.html
4. <https://drive.google.com/file/d/13jgqjSY-GOKhLoOsedUuZPXHTzQXrnt1/view?usp=sharing>
5. https://drive.google.com/file/d/1Gp_f7SmzDBMuzIxSpnR948F1KHKApHM1/view?usp=sharing
6. <https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>
7. https://tatsu-lab.github.io/alpaca_eval/
8. https://github.com/rasbt/LLMs-from-scratch/blob/main/ch07/01_main-chapter-code/instruction-data.json
9. <https://github.com/openai/tiktoken>
10. <https://arxiv.org/abs/2405.14394>
11. <https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct>
12. <https://arxiv.org/pdf/2009.03300>
13. https://drive.google.com/file/d/1OCpr4GZVmNNEXDpLuG2GZ1LlnJ_uvqPN/view?usp=sharing
14. https://github.com/tatsu-lab/stanford_alpaca

Footnotes

1. Video: *Lecture 2* @ 02:03
2. Video: *Lecture 2* @ 02:45
3. Video: *Lecture 1* @ 03:21
4. Video: *Lecture 2* @ 03:50
5. Video: *Lecture 3* @ 03:25
6. Video: *Lecture 3* @ 04:09
7. Video: *Lecture 2* @ 21:19
8. Video: *Lecture 2* @ 23:23
9. Video: *Lecture 2* @ 24:04
10. Video: *Lecture 4* @ 14:22
11. Video: *Lecture 5* @ 15:30
12. Video: *Lecture 4* @ 22:09
13. Video: *Lecture 5* @ 25:10
14. Video: *Lecture 5* @ 25:59
15. Video: *Lecture 3* @ 02:07
16. Video: *Lecture 3* @ 10:19
17. Video: *Lecture 5* @ 27:23
18. Video: *Lecture 3* @ 05:30
19. Video: *Lecture 3* @ 05:45
20. Video: *Lecture 3* @ 18:37
21. Video: *Lecture 4* @ 01:10
22. Video: *Lecture 3* @ 24:06
23. Video: *Lecture 3* @ 25:29
24. Video: *Lecture 5* @ 10:55
25. Video: *Lecture 5* @ 11:37
26. Video: *Lecture 5* @ 12:18
27. Video: *Lecture 5* @ 38:22
28. Video: *Lecture 5* @ 41:13
29. Video: *Lecture 6* @ 07:34
30. Video: *Lecture 6* @ 05:10
31. Video: *Lecture 6* @ 05:30
32. Video: *Lecture 6* @ 04:30
33. Video: *Lecture 5* @ 30:49
34. Video: *Code an LLM Tokenizer from Scratch in Python* @ 01:23
35. Video: *Code an LLM Tokenizer from Scratch in Python* @ 03:50
36. Video: *Code an LLM Tokenizer from Scratch in Python* @ 00:41
37. Video: *Code an LLM Tokenizer from Scratch in Python* @ 13:35
38. Video: *The entire Data Preprocessing Pipeline of Large Language Models (LLMs)* @ 30:23

- 39. Video: *Code an LLM Tokenizer from Scratch in Python* @ 44:32
- 40. Video: *The GPT Tokenizer: Byte Pair Encoding* @ 05:58
- 41. Video: *The GPT Tokenizer: Byte Pair Encoding* @ 07:19
- 42. Video: *The GPT Tokenizer: Byte Pair Encoding* @ 09:23
- 43. Video: *The GPT Tokenizer: Byte Pair Encoding* @ 12:08
- 44. Video: *The GPT Tokenizer: Byte Pair Encoding* @ 17:01
- 45. Video: *The GPT Tokenizer: Byte Pair Encoding* @ 34:07
- 46. Video: *Code an LLM Tokenizer from Scratch in Python* @ 25:20
- 47. Video: *Code an LLM Tokenizer from Scratch in Python* @ 26:42
- 48. Video: *Code an LLM Tokenizer from Scratch in Python* @ 60:54
- 49. Video: *Code an LLM Tokenizer from Scratch in Python* @ 47:54
- 50. Video: *Code an LLM Tokenizer from Scratch in Python* @ 59:31
- 51. Video: *Creating Input-Target data pairs using Python DataLoader* @ 06:08
- 52. Video: *Creating Input-Target data pairs using Python DataLoader* @ 14:29
- 53. Video: *Creating Input-Target data pairs using Python DataLoader* @ 21:18
- 54. Video: *Creating Input-Target data pairs using Python DataLoader* @ 43:17
- 55. Video: *Creating Input-Target data pairs using Python DataLoader* @ 50:51
- 56. Video: *What are token embeddings?* @ 00:41
- 57. Video: *What are token embeddings?* @ 05:29
- 58. Video: *What are token embeddings?* @ 43:20
- 59. Video: *What are token embeddings?* @ 31:36
- 60. Video: *The importance of Positional Embeddings* @ 04:05
- 61. Video: *The importance of Positional Embeddings* @ 08:14
- 62. Video: *The importance of Positional Embeddings* @ 13:42
- 63. Video: *The importance of Positional Embeddings* @ 09:34
- 64. Video: *The entire Data Preprocessing Pipeline of Large Language Models (LLMs)* @ 02:05
- 65. Video: *The entire Data Preprocessing Pipeline of Large Language Models (LLMs)* @ 13:48
- 66. Video: *The entire Data Preprocessing Pipeline of Large Language Models (LLMs)* @ 52:51
- 67. Video: *The entire Data Preprocessing Pipeline of Large Language Models (LLMs)* @ 64:02
- 68. Video: *The entire Data Preprocessing Pipeline of Large Language Models (LLMs)* @ 83:10
- 69. Video: *The entire Data Preprocessing Pipeline of Large Language Models (LLMs)* @ 93:34
- 70. Video: *Lecture 13* @ 15:09
- 71. Video: *Lecture 13* @ 03:26
- 72. Video: *Lecture 13* @ 30:19
- 73. Video: *Lecture 13* @ 37:10
- 74. Video: *Lecture 14* @ 05:28
- 75. Video: *Lecture 14* @ 09:33
- 76. Video: *Lecture 14* @ 20:36
- 77. Video: *Lecture 14* @ 26:48
- 78. Video: *Lecture 14* @ 45:18
- 79. Video: *Lecture 15* @ 09:04

80. Video: *Lecture 15 @ 75:56*
81. Video: *Lecture 15 @ 43:39*
82. Video: *Lecture 15 @ 50:33*
83. Video: *Lecture 16 @ 09:43*
84. Video: *Lecture 16 @ 28:25*
85. Video: *Lecture 16 @ 33:59*
86. Video: *Lecture 13 @ 08:17*
87. Video: *Lecture 17 @ 20:44*
88. Video: *Lecture 18 @ 02:45*
89. Video: *Lecture 18 @ 19:25*
90. Video: *Lecture 18 @ 25:38*
91. Video: *Lecture 18 @ 51:56*
92. Video: *Lecture 16 @ 51:59*
93. Video: *Lecture 14 @ 36:26*
94. Video: *Lecture 15 @ 46:25*
95. Video: *Birds Eye View of the LLM Architecture @ 12:18*
96. Video: *Birds Eye View of the LLM Architecture @ 00:00*
97. Video: *Birds Eye View of the LLM Architecture @ 12:40*
98. Video: *Coding the entire LLM Transformer Block @ 12:18*
99. Video: *Coding the entire LLM Transformer Block @ 17:17*
100. Video: *Shortcut connections in the LLM Architecture @ 07:35*
101. Video: *Layer Normalization in the LLM Architecture @ 37:42*
102. Video: *Layer Normalization in the LLM Architecture @ 30:04*
103. Video: *Coding the entire LLM Transformer Block @ 34:24*
104. Video: *GELU Activation Function in the LLM Architecture @ 17:52*
105. Video: *GELU Activation Function in the LLM Architecture @ 16:30*
106. Video: *GELU Activation Function in the LLM Architecture @ 10:17*
107. Video: *Shortcut connections in the LLM Architecture @ 13:05*
108. Video: *Coding the 124 million parameter GPT-2 model @ 08:19*
109. Video: *Coding the 124 million parameter GPT-2 model @ 56:30*
110. Video: *Coding GPT-2 to predict the next token @ 01:22*
111. Video: *Coding GPT-2 to predict the next token @ 31:46*
112. Video: *Coding GPT-2 to predict the next token @ 10:21*
113. Video: *Coding GPT-2 to predict the next token @ 36:39*
114. Video: *Measuring the LLM loss function @ 02:43*
115. Video: *Measuring the LLM loss function @ 08:14*
116. Video: *Measuring the LLM loss function @ 09:36*
117. Video: *Temperature Scaling in Large Language Models (LLMs) @ 08:17*
118. Video: *Measuring the LLM loss function @ 31:37*
119. Video: *Measuring the LLM loss function @ 41:48*
120. Video: *Evaluating LLM performance on real dataset @ 04:06*

- 121. Video: *Evaluating LLM performance on real dataset* @ 05:28
- 122. Video: *Evaluating LLM performance on real dataset* @ 11:40
- 123. Video: *Evaluating LLM performance on real dataset* @ 50:51
- 124. Video: *Coding the entire LLM Pre-training Loop* @ 12:22, 32:30
- 125. Video: *Coding the entire LLM Pre-training Loop* @ 35:59, 39:28
- 126. Video: *Temperature Scaling in Large Language Models (LLMs)* @ 15:10, 22:47
- 127. Video: *Top-k sampling in Large Language Models* @ 06:15
- 128. Video: *Loading pre-trained weights from OpenAI GPT-2* @ 03:31
- 129. Video: *Loading pre-trained weights from OpenAI GPT-2* @ 08:23
- 130. Video: *Loading pre-trained weights from OpenAI GPT-2* @ 14:33, 36:31
- 131. Video: *Loading pre-trained weights from OpenAI GPT-2* @ 41:22
- 132. Video: *Saving and loading LLM model weights using PyTorch* @ 03:28
- 133. Video: *Saving and loading LLM model weights using PyTorch* @ 07:32
- 134. Video: *Saving and loading LLM model weights using PyTorch* @ 10:19
- 135. Video: *Introduction to LLM Finetuning* @ 00:42
- 136. Video: *Introduction to LLM Finetuning* @ 02:06
- 137. Video: *Introduction to LLM Finetuning* @ 09:44
- 138. Video: *Introduction to LLM Finetuning* @ 16:38
- 139. Video: *Introduction to LLM Finetuning* @ 19:24
- 140. Video: *Introduction to LLM Finetuning* @ 20:04
- 141. Video: *Introduction to LLM Finetuning* @ 22:09
- 142. Video: *Dataloaders in LLM Classification Finetuning* @ 09:32
- 143. Video: *Dataloaders in LLM Classification Finetuning* @ 10:54
- 144. Video: *Dataloaders in LLM Classification Finetuning* @ 14:23
- 145. Video: *Dataloaders in LLM Classification Finetuning* @ 25:26
- 146. Video: *Coding the model architecture* @ 18:54
- 147. Video: *Coding the model architecture* @ 21:03
- 148. Video: *Coding the model architecture* @ 23:15
- 149. Video: *Coding the model architecture* @ 24:06
- 150. Video: *Coding a fine-tuned LLM* @ 17:18
- 151. Video: *Coding a fine-tuned LLM* @ 27:41
- 152. Video: *Coding a fine-tuned LLM* @ 15:12
- 153. Video: *Coding a fine-tuned LLM* @ 36:48
- 154. Video: *Coding a fine-tuned LLM* @ 41:38
- 155. Video: *Coding a fine-tuned LLM* @ 46:54
- 156. Video: *Introduction to LLM Finetuning* @ 12:31
- 157. Video: *Introduction to LLM Instruction Fine-tuning* @ 02:06
- 158. Video: *Introduction to LLM Instruction Fine-tuning* @ 05:34
- 159. Video: *Introduction to LLM Instruction Fine-tuning* @ 04:11
- 160. Video: *Introduction to LLM Instruction Fine-tuning* @ 10:25
- 161. Video: *Introduction to LLM Instruction Fine-tuning* @ 09:44

- 162. Video: *Introduction to LLM Instruction Fine-tuning* @ 14:33
- 163. Video: *Data Batching in LLM instruction fine-tuning* @ 06:11
- 164. Video: *Data Batching in LLM instruction fine-tuning* @ 22:40
- 165. Video: *Data Batching in LLM instruction fine-tuning* @ 20:36
- 166. Video: *Dataloaders in Instruction Fine-tuning* @ 00:41
- 167. Video: *Dataloaders in Instruction Fine-tuning* @ 19:19
- 168. Video: *Instruction fine-tuning: Loading pre-trained LLM weights* @ 05:35
- 169. Video: *Instruction fine-tuning: Loading pre-trained LLM weights* @ 06:56
- 170. Video: *Instruction fine-tuning: Loading pre-trained LLM weights* @ 04:13
- 171. Video: *LLM fine-tuning training loop* @ 13:14
- 172. Video: *LLM fine-tuning training loop* @ 20:47
- 173. Video: *Evaluating fine-tuned LLM using Ollama* @ 05:29
- 174. Video: *Evaluating fine-tuned LLM using Ollama* @ 13:09
- 175. Video: *Evaluating fine-tuned LLM using Ollama* @ 15:54
- 176. Video: *Evaluating fine-tuned LLM using Ollama* @ 16:36
- 177. Video: *Build LLMs from scratch 20 minutes summary* @ 06:54
- 178. Video: *Build LLMs from scratch 20 minutes summary* @ 11:03
- 179. Video: *Build LLMs from scratch 20 minutes summary* @ 15:53
- 180. Video: *Build LLMs from scratch 20 minutes summary* @ 18:39